

Estrutura de Dados 2

Prof. Igor Calebe Zadi
igor.zadi@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo
Campus Catanduva



III. EDs Não-Lineares



III. EDs Não-Lineares

1. Introdução às Estruturas Não-Lineares
2. Conceitos e Terminologia
3. Representações de Árvores
4. Árvores Binárias
5. Percursos em Árvores Binárias
6. Árvores de Busca Binária (BST)
7. Árvores Balanceadas
8. Árvores B e B+
9. Comparações e Aplicações

8. Árvores B e B+



8. Árvore B - Situação Problema

- Memória Principal
 - Volátil e limitada
- Aplicações
 - Grandes quantidades de informação
 - Armazenamento permanente de informações
- Chaves mantidas em memória secundária



8. Árvore B - Situação Problema

- Árvores de Busca Binária
 - Apropriada para memória principal
 - Ineficiente em memória secundária
 - Acesso: cerca de $\log_2 n$ passos
 - Grande quantidade de acessos a disco
 - Acesso feito em blocos



8. Árvore B - Situação Problema

- Necessidade
 - Reduzir o número de acessos a disco
- Solução
 - Agrupar várias chaves dentro de um nó
 - Obter com o mesmo acesso várias chaves
 - Reduzir o número de acessos
 - Diminuir o tempo necessário para inserções, remoções e pesquisas.

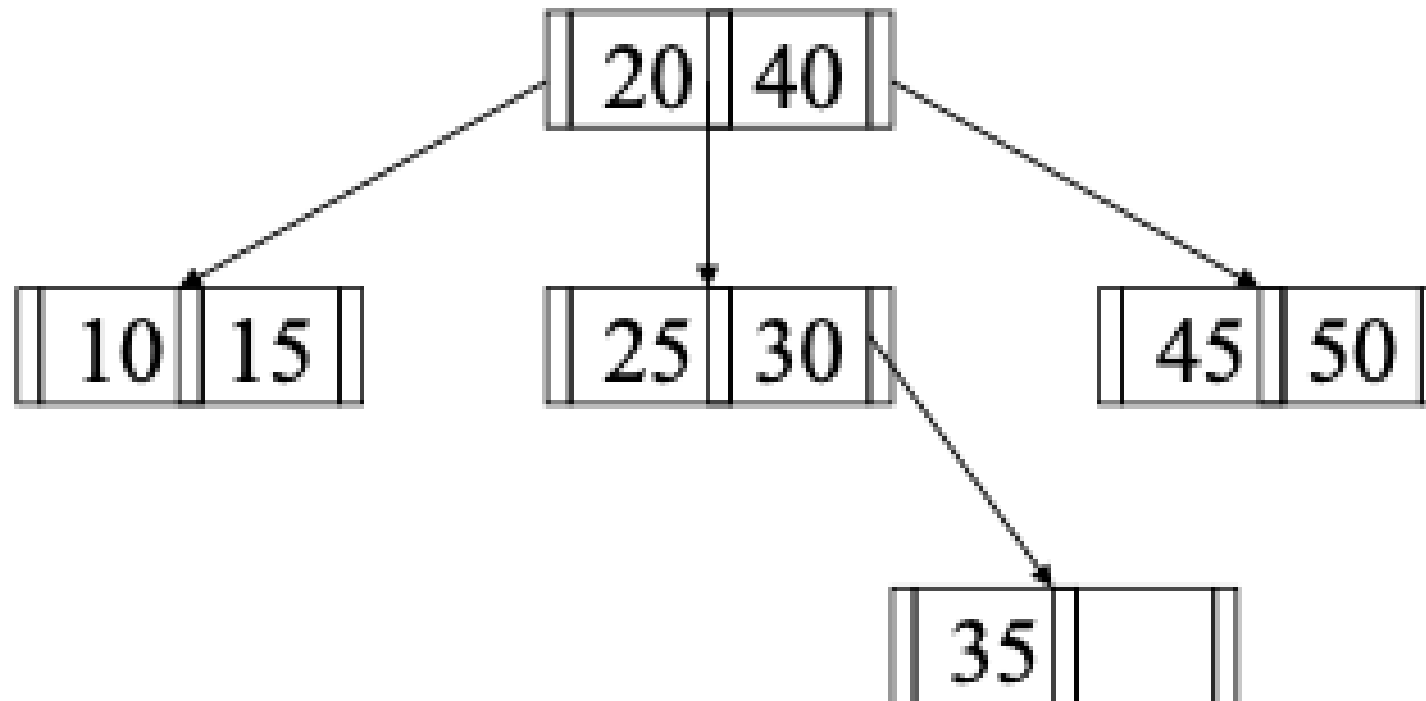


8. Árvores Multivias ou M-Vias

- Definição
 - Uma árvore de busca multivias de ordem **M** é uma árvore *n-ária* na qual todos os nós têm **grau** menor ou igual a **M**.
 - Um nó com **M** descendentes contém **M-1** valores de chave.

8. Árvores Multivias ou M-Vias

- Exemplo: $M = 3$



8. Árvores Multivias ou M-Vias

- Desempenho da Busca
 - Árvores multivias de **N** chaves e fator de ramificação **S** (quantidade média de filhos de cada nó).
 - Caminho médio de busca: $O(\log_S N)$
 - Se **N = 10^6** e **S = 100**, então uma busca requer, em média, $\log_{100} 10^6 = 3$ passos.
 - Árvore de busca binária: $\log_2 10^6 = 20$ passos.



8. Árvores Multivias ou M-Vias

- Problema
 - Inserções aleatórias de maneira irrestrita
 - Aumento do caminho de busca
- Solução
 - Balanceamento



8. Árvore B - Definição

- Bayer e McCreight em 1970
- Uma **árvore B** de ordem **M** é uma árvore de busca multivias **balanceada**
- Uma árvore B ou está vazia ou possui nós com **K** apontadores e **K-1** chaves.
- Um nó de uma **árvore B** é chamado de página.



8. Árvore B - Definição

- Chave:
 - É o valor que identifica, ordena e direciona a busca dentro da árvore.
 - É o elemento usado para comparar, inserir, remover e navegar pelos nós.
 - Representa o critério de comparação usado para:
 - localizar um item,
 - direcionar para uma subárvore,
 - decidir onde inserir ou remover valores.



8. Árvore B - Características Estruturais

- Na raiz, **K** deve ser, no mínimo, 2.
 - Ou seja, a raiz possui no mínimo dois filhos e uma chave.
- Nos demais nós, **K** deve ser, no mínimo, **M/2**.
 - Ou seja, os demais nós possuem, no mínimo, **M/2** filhos e **M/2-1** chaves;
- Exceção: Folhas não têm filhos.



8. Árvore B - Características Estruturais

- O valor máximo de **K** é **M**
 - Ou seja, todos os nós têm, no máximo, **M-1** chaves e **M** filhos;
- Todas as folhas estão no mesmo nível (balanceamento).
- **M** deve ser escolhido de forma que o número máximo de chaves nos nós da árvore seja uma **potência de 2**

8. Árvore B - Características Estruturais

- Formato do nó:
 - $N, A_0, (C_1A_1), (C_2A_2), \dots, (C_{M-1}A_{M-1})$ onde:
 - $N, M/2 \leq N \leq M$, é o número de entrada ativas (ocupadas) de um nó em um dado momento;
 - $A_i, 0 \leq i \leq M-1$, é um apontador para uma subárvore;
 - $C_i, 1 \leq i \leq M-1$, é um valor de chave e $C_i < C_{i+1}$
 - O par (C_iA_i) é **chamado de entrada**
 - O apontador A_0 também é definido como **entrada**

8. Árvore B - Características Estruturais

- Definição do Nó

```
typedef char Tipo;
```

```
struct Noh {  
    int n;  
    Tipo chv[M-1];  
    Noh* pont[M];  
};
```

8. Árvore B - Características Estruturais

- Seja uma página com **D** chaves:
 - Para qualquer chave **y**, pertencente à página apontada por A_0 , $y < C_1$
 - Para qualquer chave **y**, pertencente à página apontada por A_i , $1 \leq i \leq D-1$,
 $C_i < \mathbf{y} < C_{i+1}$
 - Para qualquer chave **y**, pertencente à página apontada por A_D , $y > C_D$

8. Árvore B - Características Estruturais

- Por exemplo, uma árvore de ordem $M = 5$, um nó poderia ser:

[23 | 52 | 77 | 89]

- Com 5 ponteiros A_0, A_1, A_2, A_3 e A_4
- Significa que:
 - $A_0 \rightarrow$ contém chaves < 23
 - $A_1 \rightarrow$ contém chaves entre 23 e 52
 - $A_2 \rightarrow$ contém chaves entre 52 e 77
 - $A_3 \rightarrow$ contém chaves entre 77 e 89
 - $A_4 \rightarrow$ contém chaves > 89



8. Árvore B - Características Estruturais

- A ordem **M** determina as quantidades máximas e mínimas de chaves dentro de cada nó.
- O número mínimo de chaves é estabelecido para determinar o percentual mínimo de ocupação dentro de um nó. Na árvore B esse percentual é de 50% (não considerando a raiz).



8. Árvore B - Inserção

- Em uma árvore B, a inserção de uma nova chave ocorre sempre em um **nó folha**.
- Passos:
 - Localizar a folha dentro da qual a chave deve ser inserida
 - Se a folha **não estiver completa**, inserir chave na ordem correta
 - Se a folha **estiver completa**, realizar a **cisão da página**.

8. Árvore B - Inserção

- Exemplo: $M = 5$
- Inserir chave 85

85			
----	--	--	--
- Inserir chave 60

60	85		
----	----	--	--
- Inserir chave 52

52	60	85	
----	----	----	--
- Inserir chave 70

52	60	70	85
----	----	----	----
- Inserir chave 58 → **Realizar cisão!**

8. Árvore B - Inserção

- Cisão de Página
 - O processo de cisão consiste em **separar** a folha **completa em duas**: folha esquerda e folha direita.
 - As M chaves serão divididas em três grupos:
 - As $(M / 2)$ chaves menores ficam na folha esquerda
 - As $(M / 2)$ chaves maiores ficam na folha direita
 - A chave do **meio** é colocada no nó **pai**, se possível.
 - OBS: A divisão é inteira

8. Árvore B - Inserção

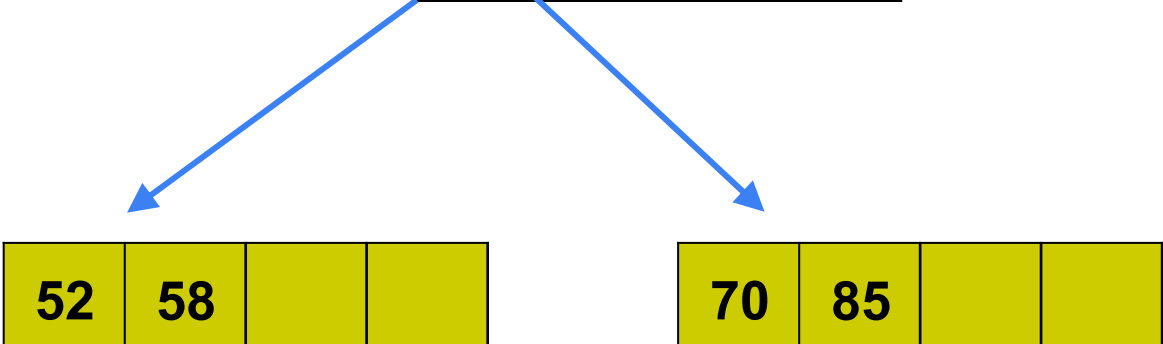
- Exemplo: $M = 5$

- Nó antes da inserção:

52	60	70	85
----	----	----	----

- Inserir chave 58:

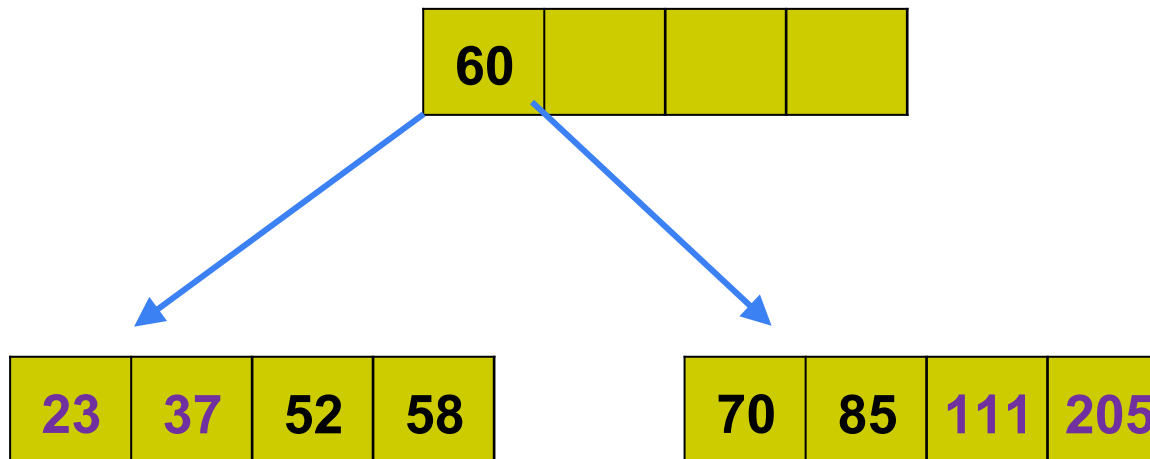
60			
----	--	--	--



```
graph TD; Root["60 | | | "] --> Leaf1["52 | 58 | | "]; Root --> Leaf2["70 | 85 | | "];
```

8. Árvore B - Inserção

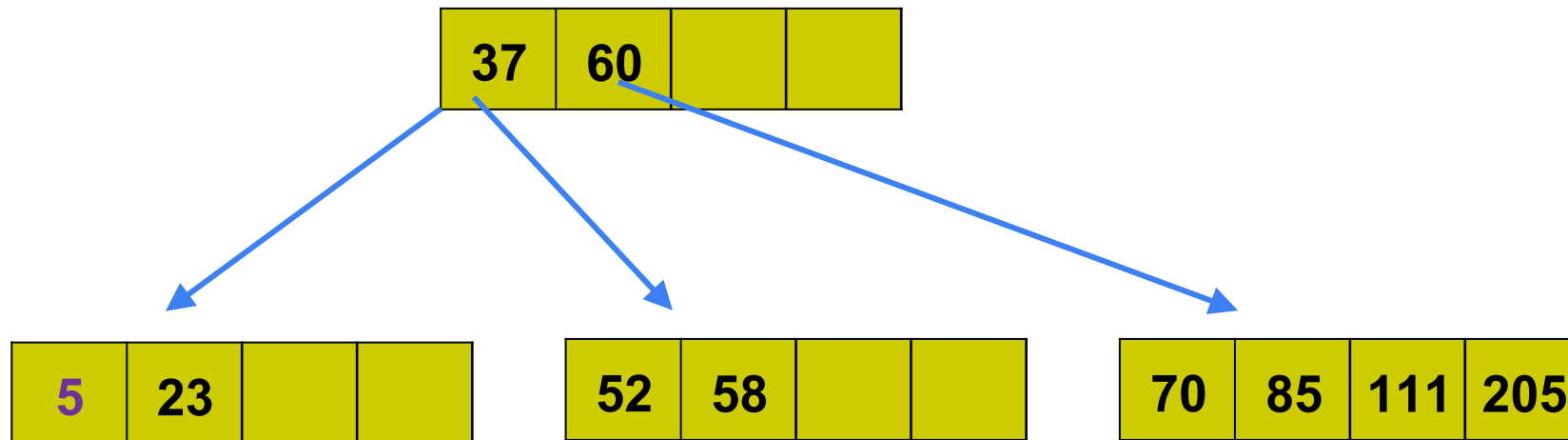
- Inserir chaves: 37, 111, 23 e 205



- Inserir chave 5 → **Realizar Cisão**

8. Árvore B - Inserção

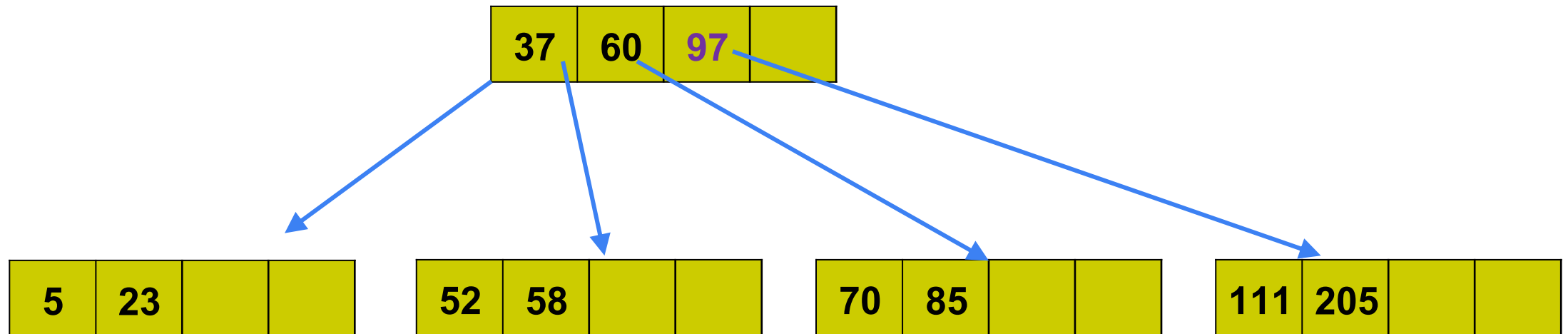
- Inserir chave 5 (depois de inserido na árvore)



- Inserir chave 97 → **Realizar Cisão**

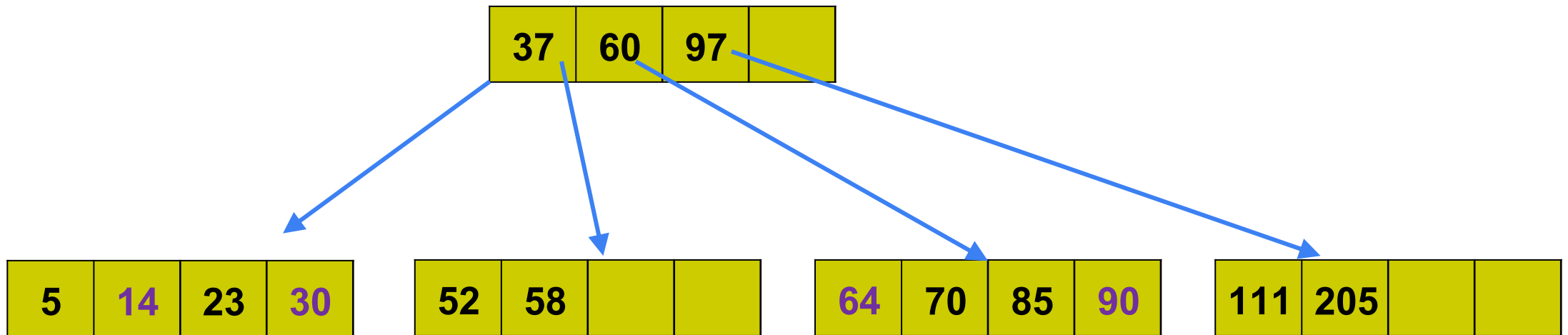
8. Árvore B - Inserção

- Inserir chave 97 (depois de inserido na árvore)



8. Árvore B - Inserção

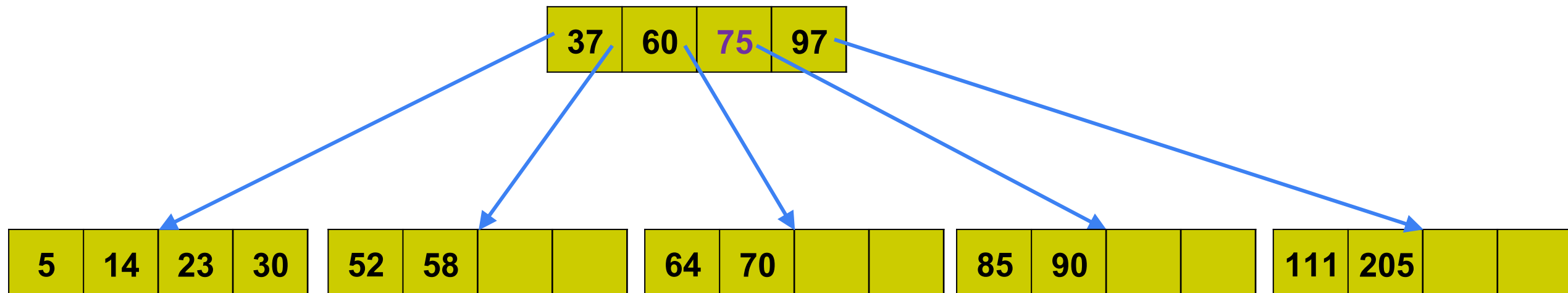
- Inserir chaves 64, 14, 90 e 30



- Inserir chave 75 → **Realizar cisão**

8. Árvore B - Inserção

- Inserir chaves 75 (depois de inserido)



- Inserir chave 25 → **Realizar cisão**

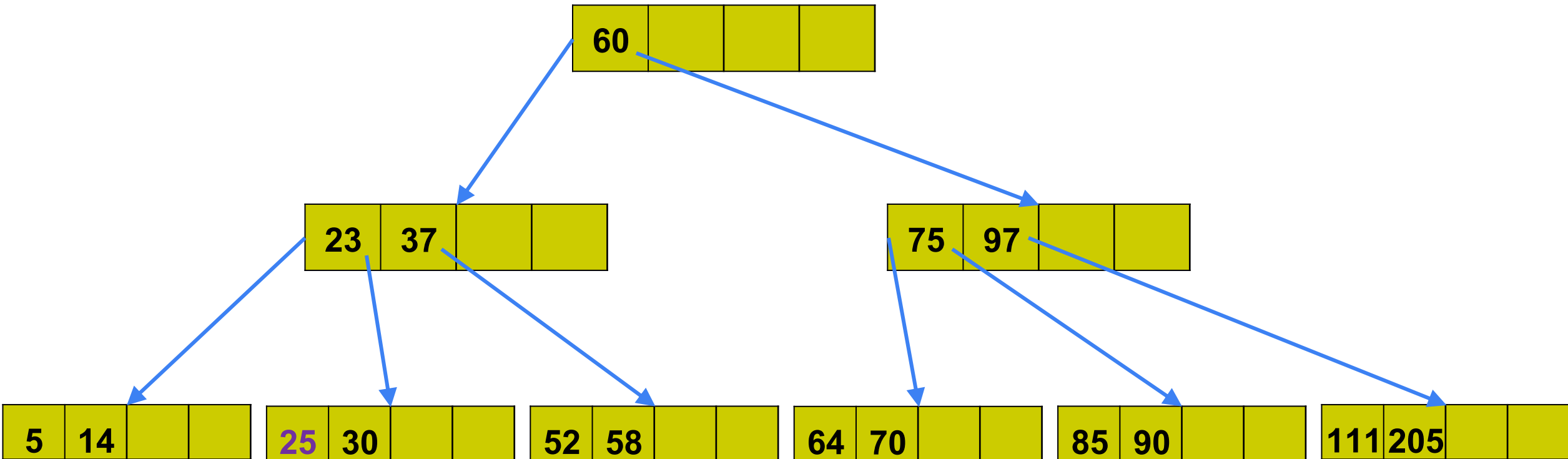


8. Árvore B - Inserção

- A inserção da nova entrada no nó pai pode acarretar a necessidade de uma nova cisão;
- A cisão de páginas é **propagável**, podendo atingir até mesmo a raiz da árvore.
- Neste caso, **surge uma nova raiz**, o que implica em alteração da altura da árvore.
- Após o processo de inserção, a árvore permanece balanceada.

8. Árvore B - Inserção

- Inserir chaves 25 (depois de inserido)



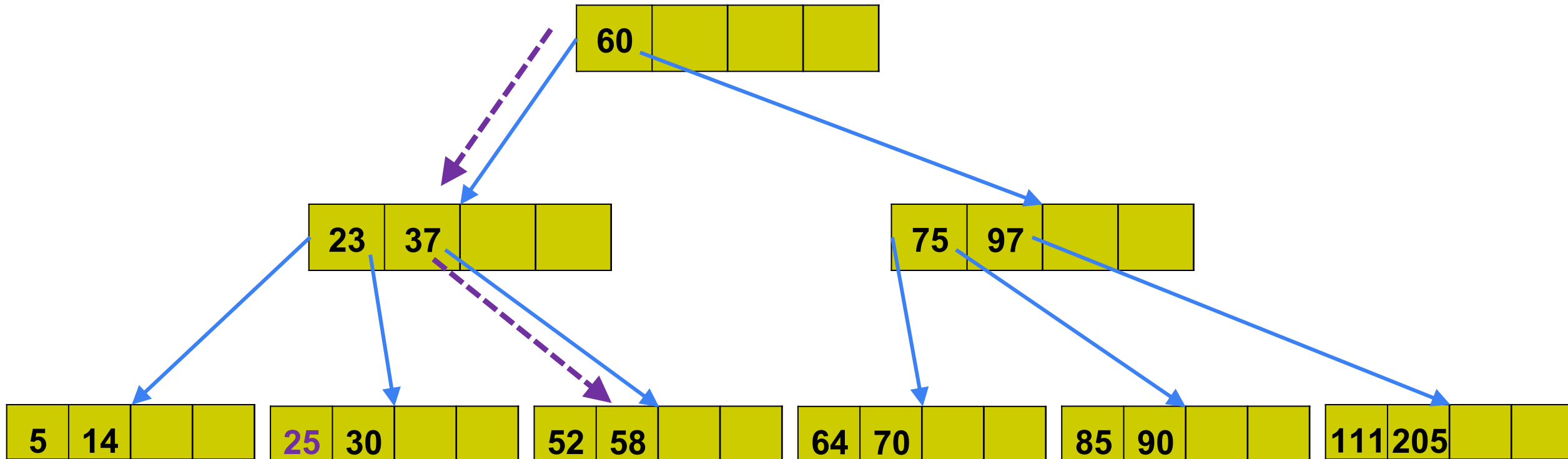


8. Árvore B - Consulta

- Verifica se a chave procurada está na raiz
- Caso não esteja, se a chave for menor que a chave C_i , $1 \leq i \leq N-1$, então repetir a pesquisa na subárvore A_{i-1} ;
- A pesquisa termina quando encontramos a chave ou um apontador A_i igual a nulo.

8. Árvore B - Consulta

- Exemplo: Procurar chave 52



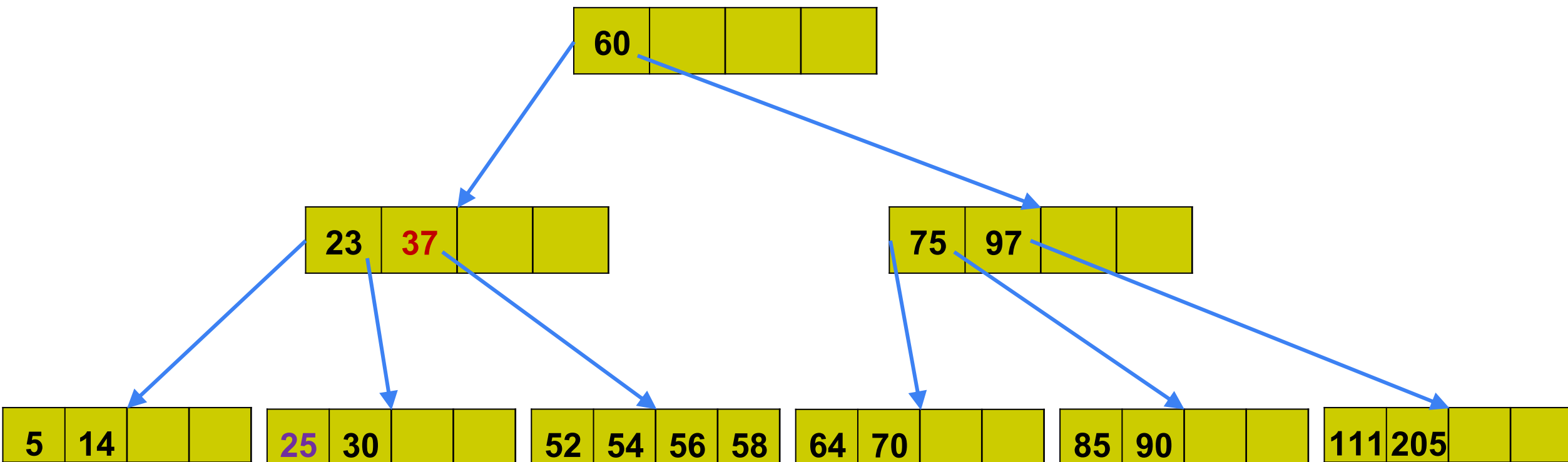


8. Árvore B - Remoção

- Remoção de uma chave **X**
 - **Caso 1**: A chave X **não** se encontra em uma folha
 - X é substituída pela chave Y, imediatamente maior
 - Y necessariamente pertence a uma folha

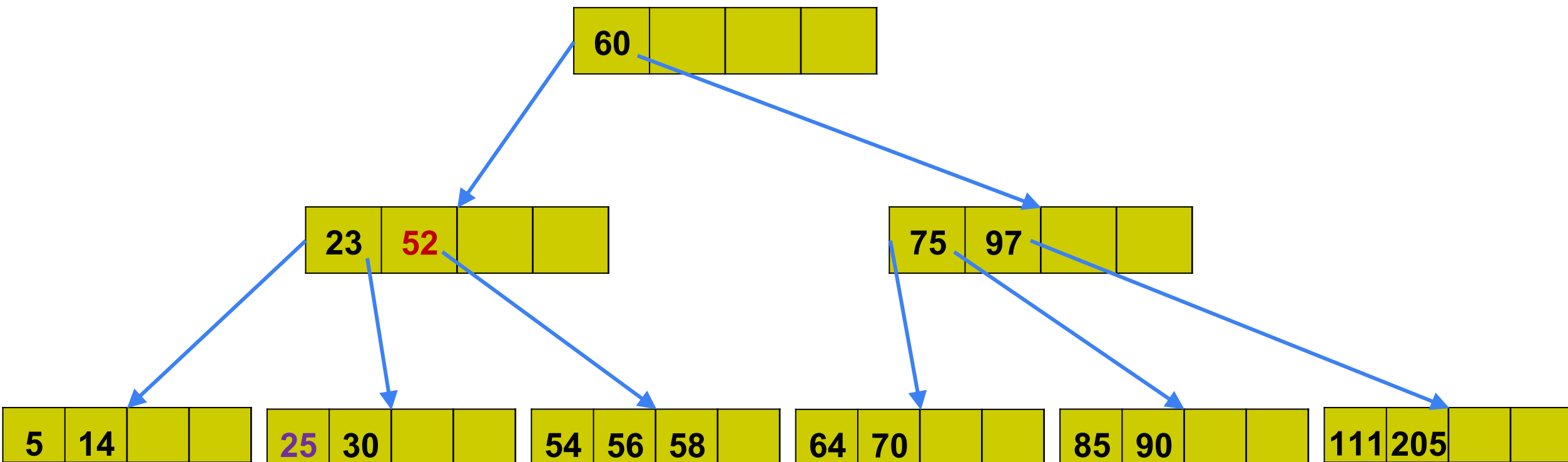
8. Árvore B - Remoção

- Exemplo: **Caso 1** – Remover a chave 37 (árvore antes)



8. Árvore B - Remoção

- Exemplo: **Caso 1** – Remover a chave 37 (árvore depois)



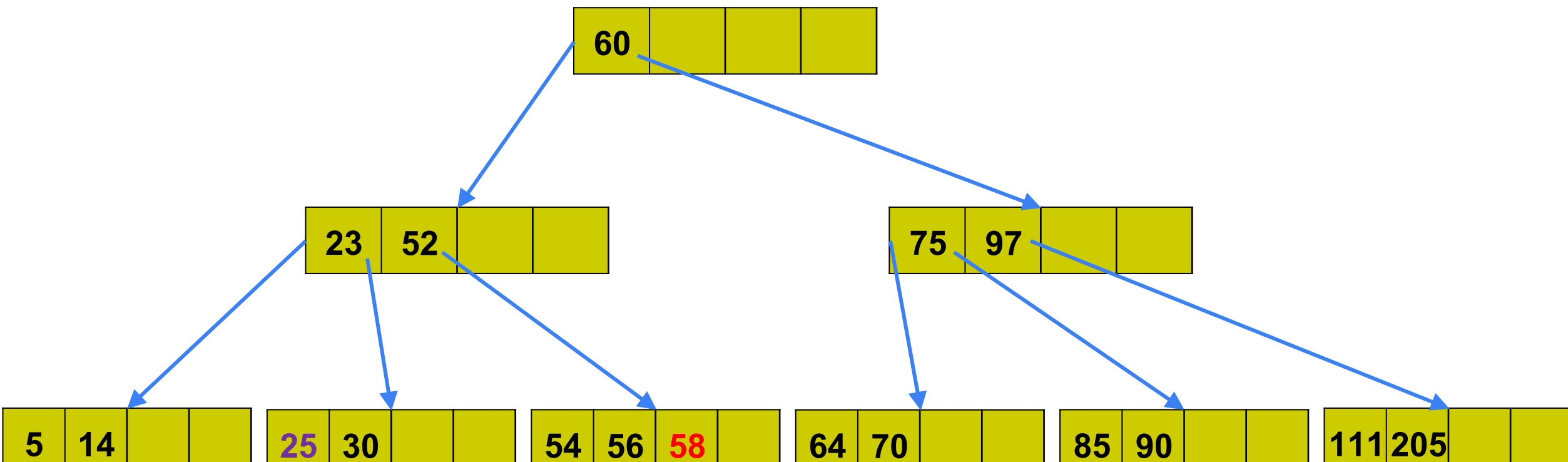


8. Árvore B - Remoção

- Remoção de uma chave **X**
 - **Caso 2:** A chave X **se encontra** em uma folha
 - A chave é simplesmente removida

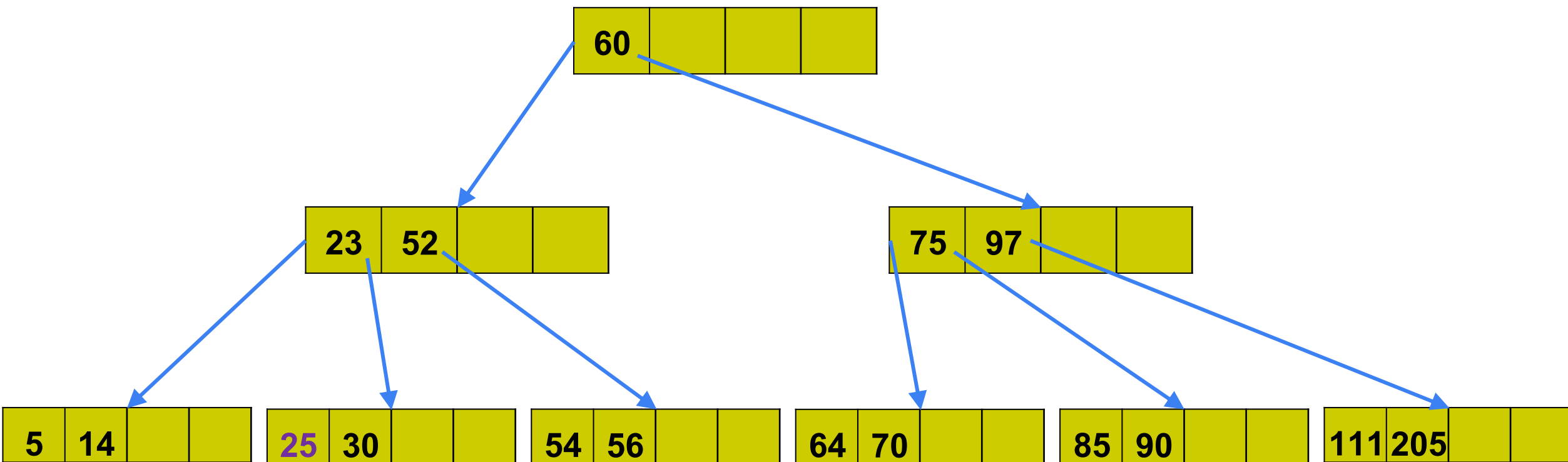
8. Árvore B - Remoção

- Exemplo: **Caso 2** – Remover a chave 58 (árvore antes)



8. Árvore B - Remoção

- Exemplo: **Caso 2** – Remover a chave 58 (árvore depois)





8. Árvore B - Remoção

- Quando uma chave é retirada de um nó folha, o número de chaves restantes pode ser menor que $(M-1)/2$
- Tratamentos:
 - Concatenação
 - Redistribuição



8. Árvore B - Remoção

- Remoção com Concatenação
 - Duas páginas **P** e **Q** são chamadas irmãos adjacentes se têm o mesmo pai **W** e são apontadas por **ponteiros adjacentes** em **W**
 - **P** e **Q** podem ser concatenadas se são irmãos adjacentes e juntas possuem menos de **M-1** chaves.

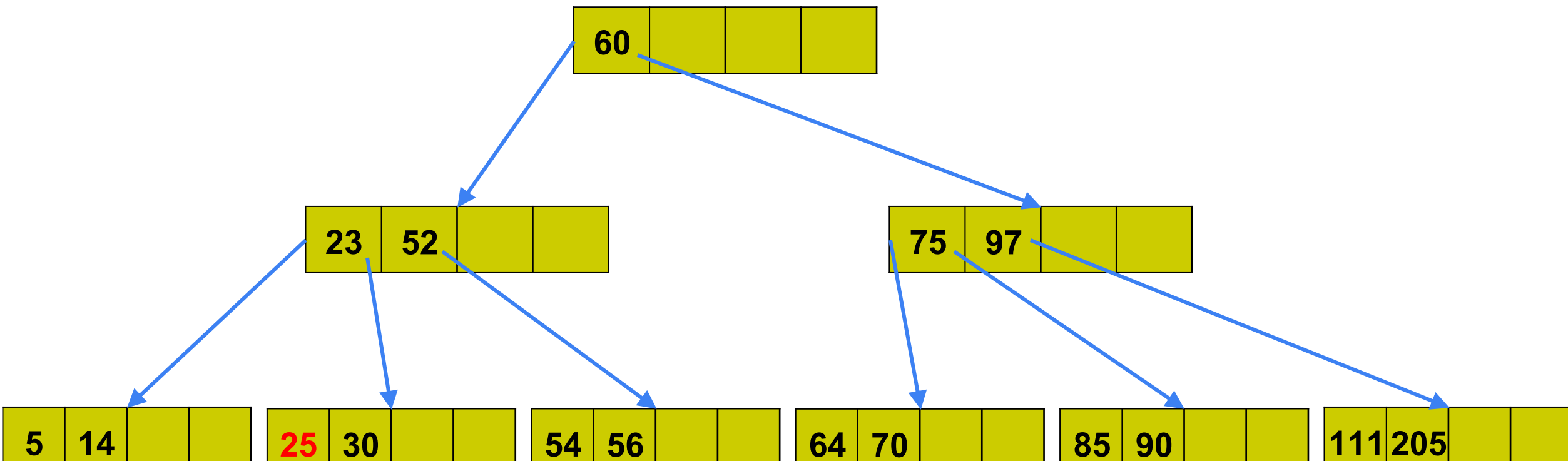


8. Árvore B - Remoção

- Remoção com Concatenação
 - A concatenação **agrupa** as entradas de duas páginas em **uma só**
 - No nó pai deixa de existir uma entrada: aquela da chave que se encontra entre os ponteiros para P e Q
 - Essa chave **passa a fazer parte do nó concatenado** e seu ponteiro desaparece

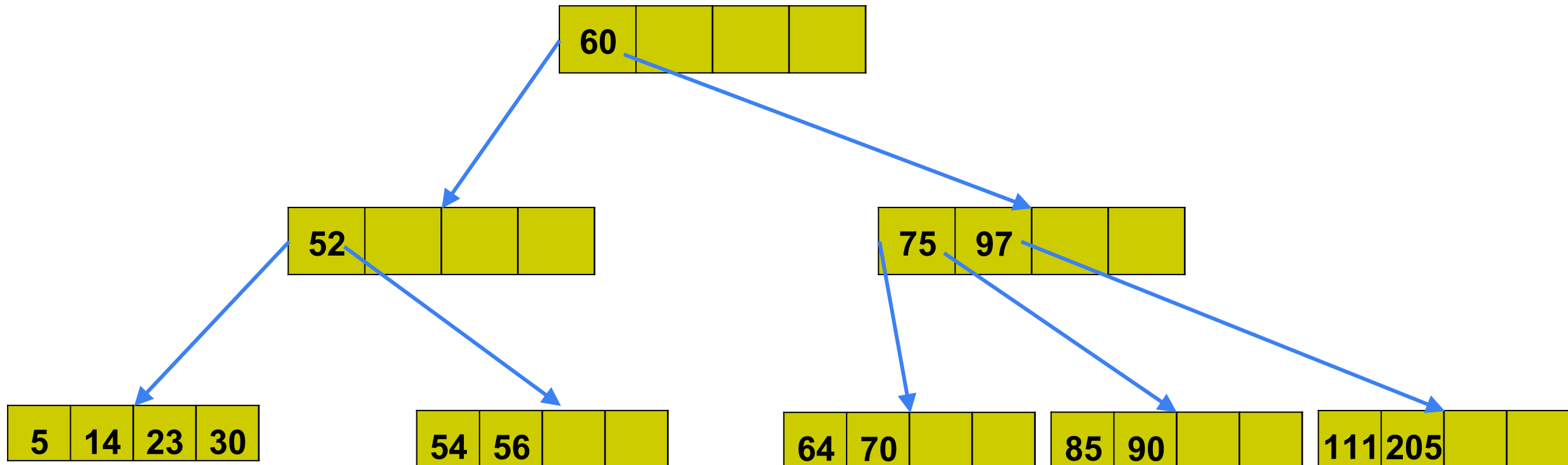
8. Árvore B - Remoção

- Exemplo: **Com concatenação** – Remover a chave 25 (antes)



8. Árvore B - Remoção

- Exemplo: **Com concatenação** – Remover a chave 25 (depois)



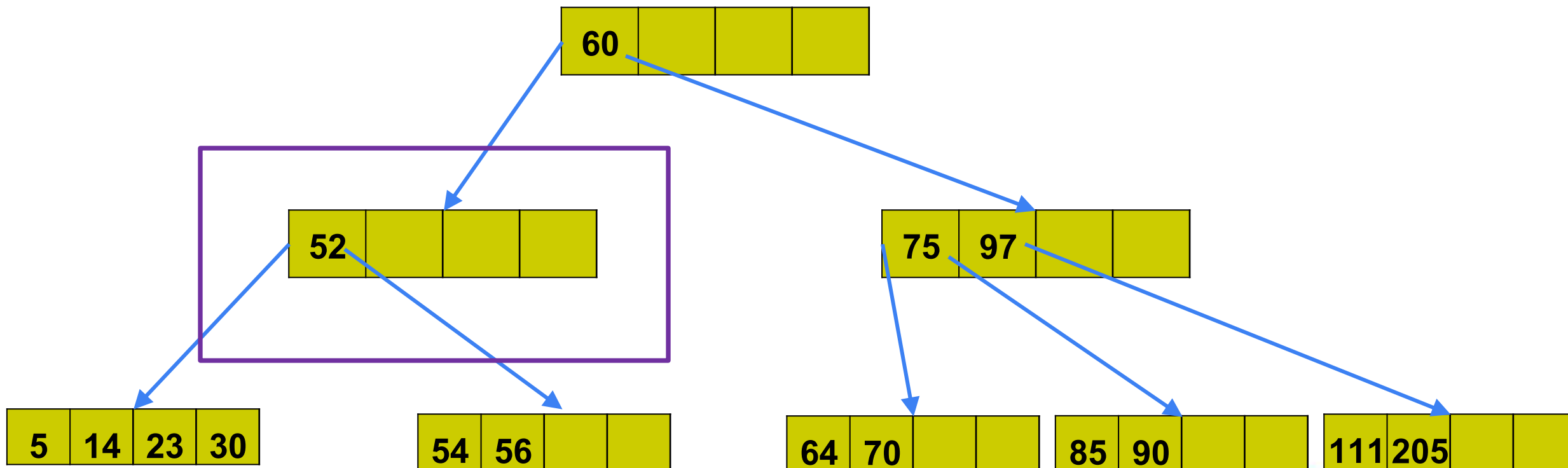


8. Árvore B - Remoção

- Remoção com Concatenação
 - Como foi retirada uma chave do nó **W**, caso ele passe a ter menos de $(M-1)/2$ chaves, o processo se repete
 - Ou seja, a concatenação é um processo propagável
 - Se a propagação atingir a raiz, **a árvore diminuirá de altura**

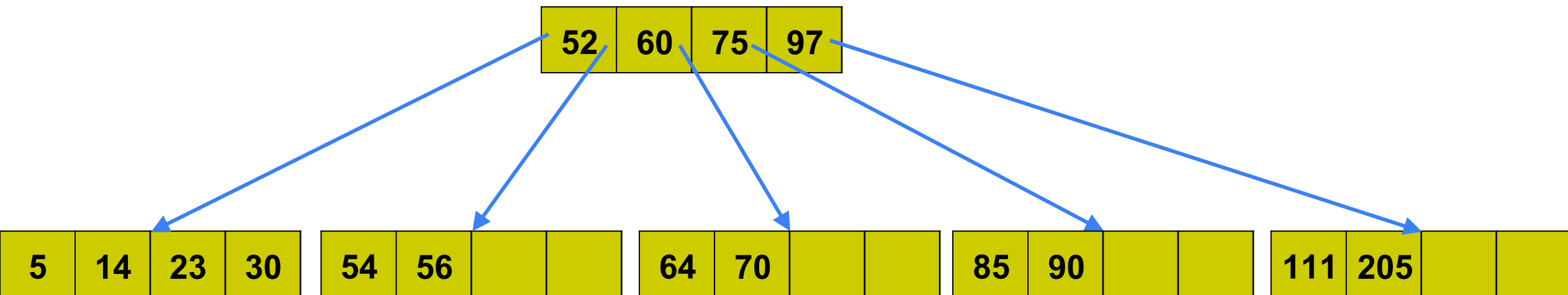
8. Árvore B - Remoção

- Exemplo: **Com concatenação** – Remover a chave 25
- Mostrar a propagação



8. Árvore B - Remoção

- Exemplo: **Com concatenação** – Remover a chave 25
- **Mostrar a propagação**



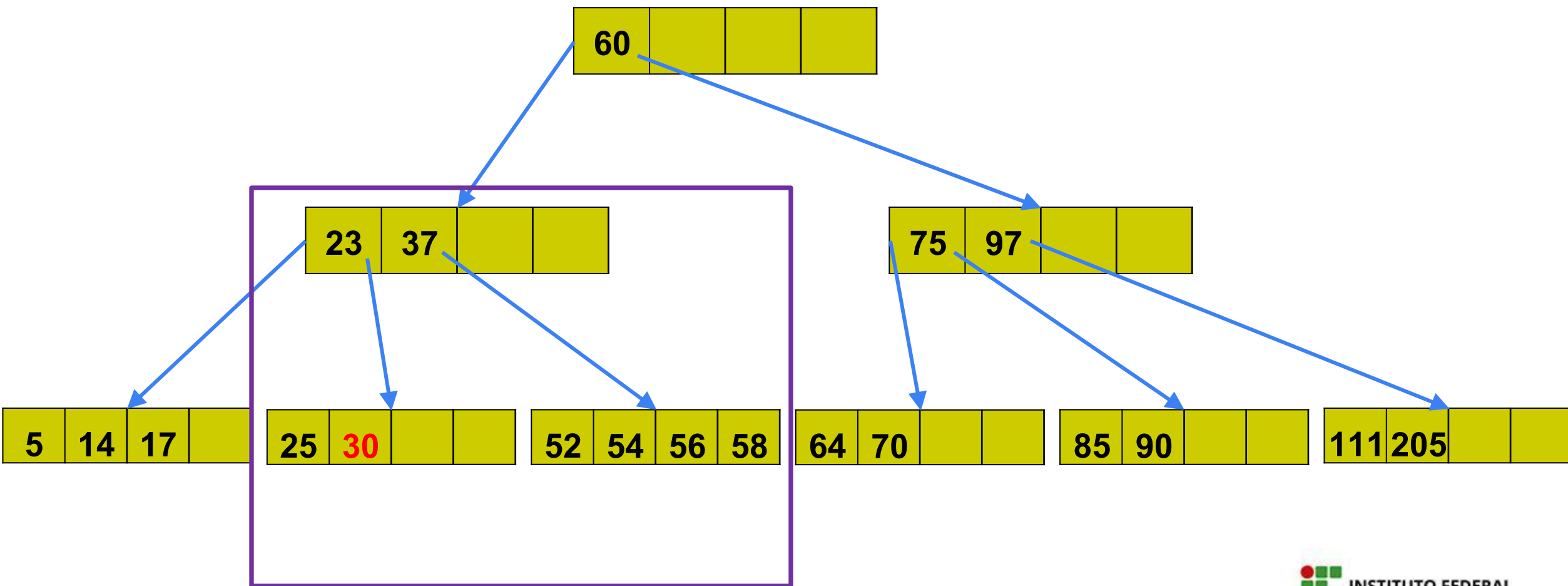


8. Árvore B - Remoção

- Remoção com Redistribuição
 - Se a página **P** e seu irmão adjacente **Q** possuem em conjunto **M-1** ou mais chaves, estas podem ser equilibradamente distribuídas:
 - Concatena-se P e Q
 - Efetua-se **a cisão** da página resultante

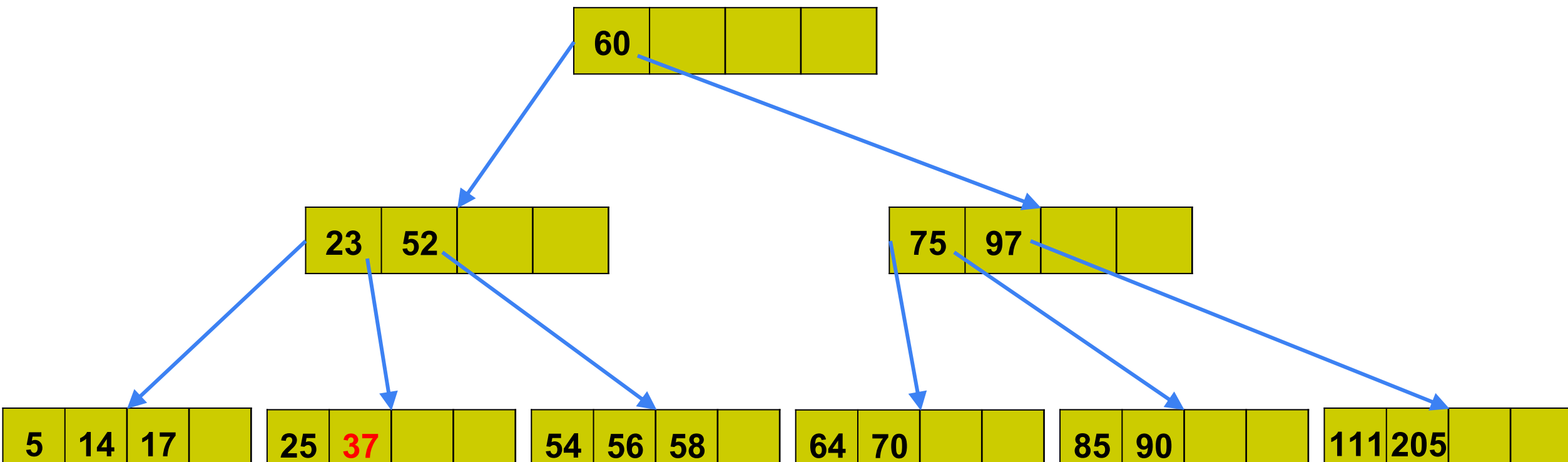
8. Árvore B - Remoção

- Exemplo: **Com redistribuição** – Remover a chave 30 (antes)



8. Árvore B - Remoção

- Exemplo: **Com redistribuição** – Remover a chave 30 (depois)





8. Árvore B - Remoção

- Remoção com Redistribuição
 - A redistribuição **não é propagável**
 - A página **W**, pai de **P** e **Q**, é modificada, mas seu número de chaves permanece o mesmo.



8. Árvore B - Remoção

- Para aplicar a remoção acompanhe sempre:
- Se $P + Q < M - 1 \rightarrow$ **CONCATENAÇÃO** (fusão + possível propagação)
- Se $P + Q \geq M - 1 \rightarrow$ **REDISTRIBUIÇÃO** (equilíbrio local, sem propagação)

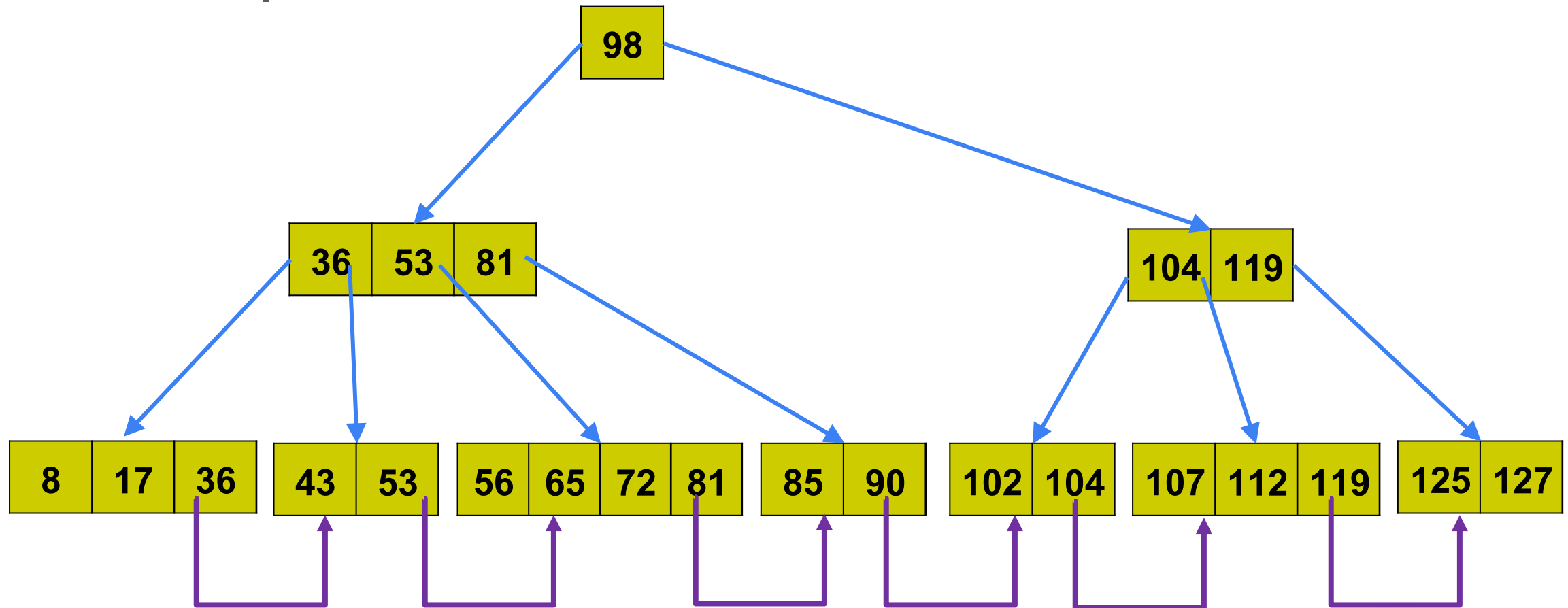


8. Árvore B+

- A árvore B+ é uma variação da estrutura básica da árvore B.
- Características:
 - **Todas as chaves são mantidas em folhas**
 - As chaves são **repetidas** em nós não-folha formando um índice
 - As folhas são ligadas oferecendo um caminho sequencial para percorrer as chaves

8. Árvore B+

- Exemplo





8. Árvore B+

- Vantagem
 - Mantém a eficiência da busca e da inserção da árvore B
 - Aumenta a eficiência da localização do próximo registro na árvore de $O(\log_2 N)$ para $O(1)$
 - Não é necessário manter nenhum ponteiro de registro em nós não-folha.



8. Árvore B+

- Utilização
 - Muitos Bancos de Dados são construídos usando o mecanismo de Árvores B+: SQLServer e Oracle;



8. Árvore B+ - Inserção

- A inserção de uma nova chave em uma árvore B+ é semelhante a inserção em uma árvore B: ocorre sempre em um nó folha.
- Passos:
 - Localizar a folha dentro da qual a chave deve ser inserida
 - Localizar a posição de inserção dentro da folha
 - Inserir a chave;
 - Se, após a inserção, a folha estiver completa, realizar a **cisão da página**.

8. Árvore B+ - Inserção

- Exemplo: $M = 5$
- Inserir chave 85

85			
----	--	--	--
- Inserir chave 60

60	85		
----	----	--	--
- Inserir chave 52

52	60	85	
----	----	----	--
- Inserir chave 70 → **Realizar cisão!**



8. Árvore B+ - Inserção

- Inserção → **Cisão de Página**
- As $M-1$ chaves serão divididas em dois grupos:
 - as $(M-1/2)$ chaves menores ficam na folha esquerda
 - as $(M-1/2)$ chaves maiores ficam na folha direita
 - A maior chave da esquerda é copiada para o nó pai.

8. Árvore B+ - Inserção

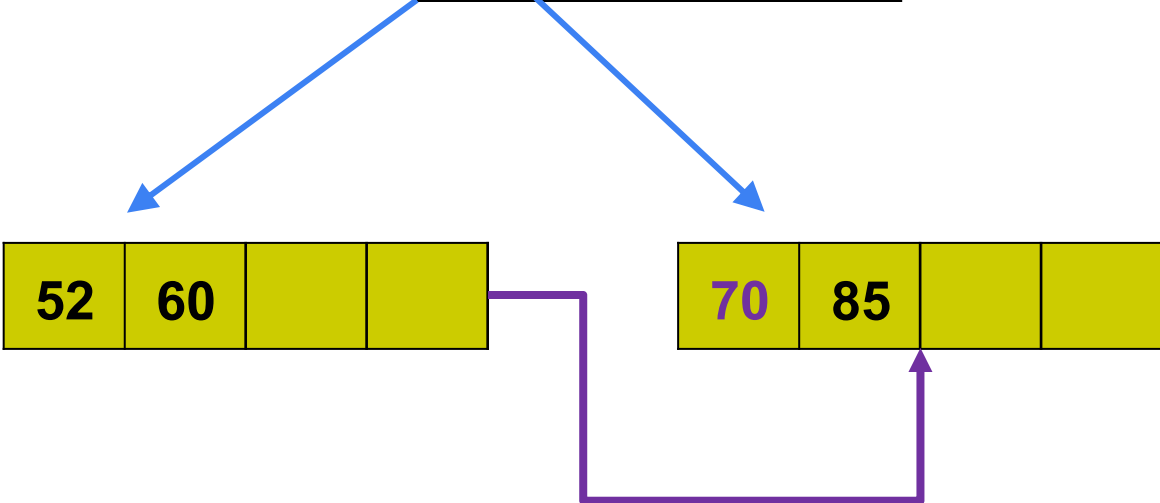
- Exemplo: $M = 5$

- Nó antes da inserção:

52	60	85	
----	----	----	--

- Inserir chave 70:

60			
----	--	--	--

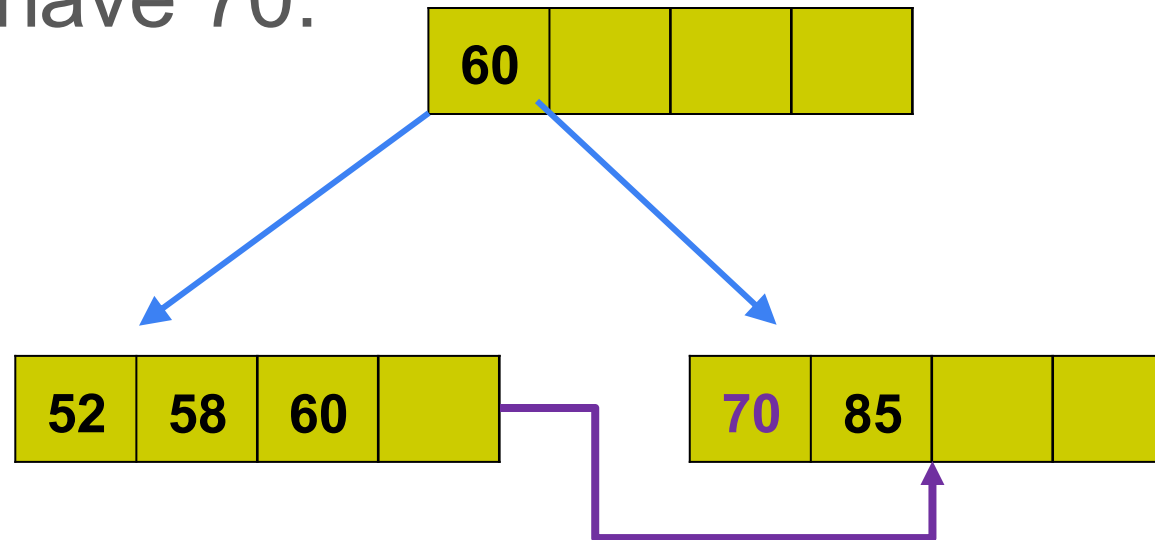


```
graph TD; Root["60 | | | "] --> Leaf1["52 | 60 | | "]; Root --> Leaf2["70 | 85 | | "]; Leaf1 --- Leaf2;
```

8. Árvore B+ - Inserção

- Exemplo: Inserir chave 58

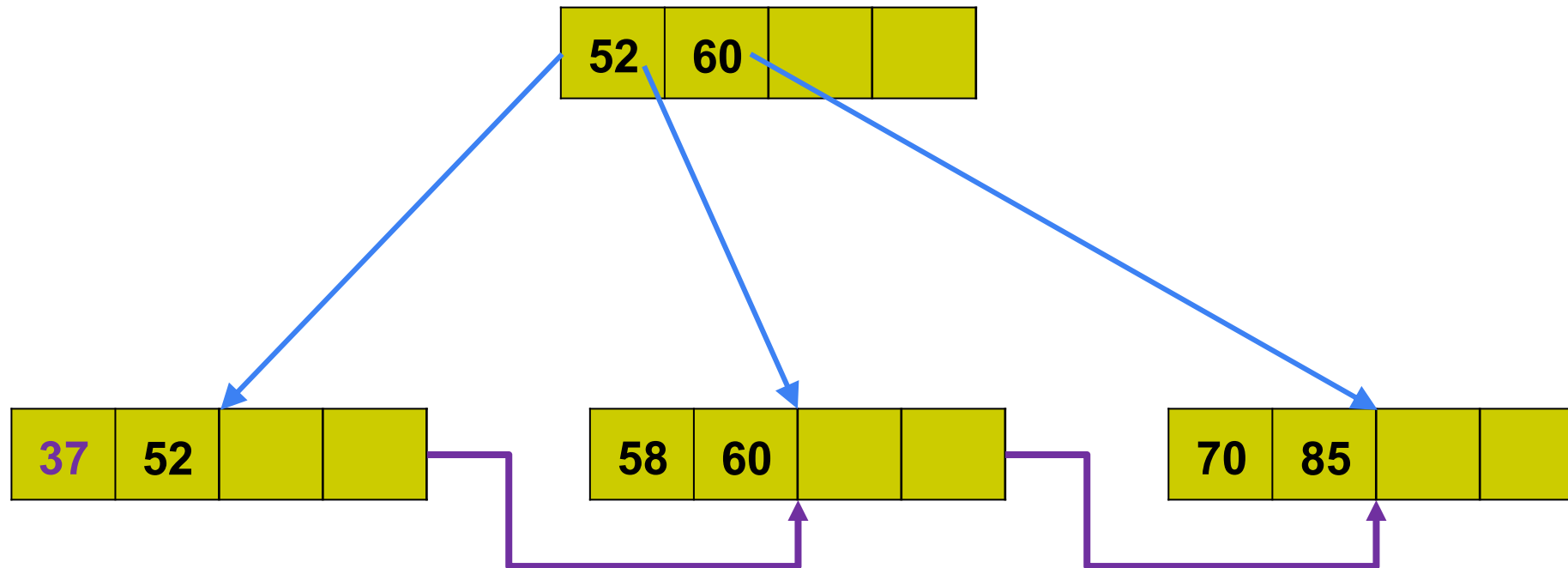
- Inserir chave 70:



- Inserir chave 37 → **Realizar cisão**

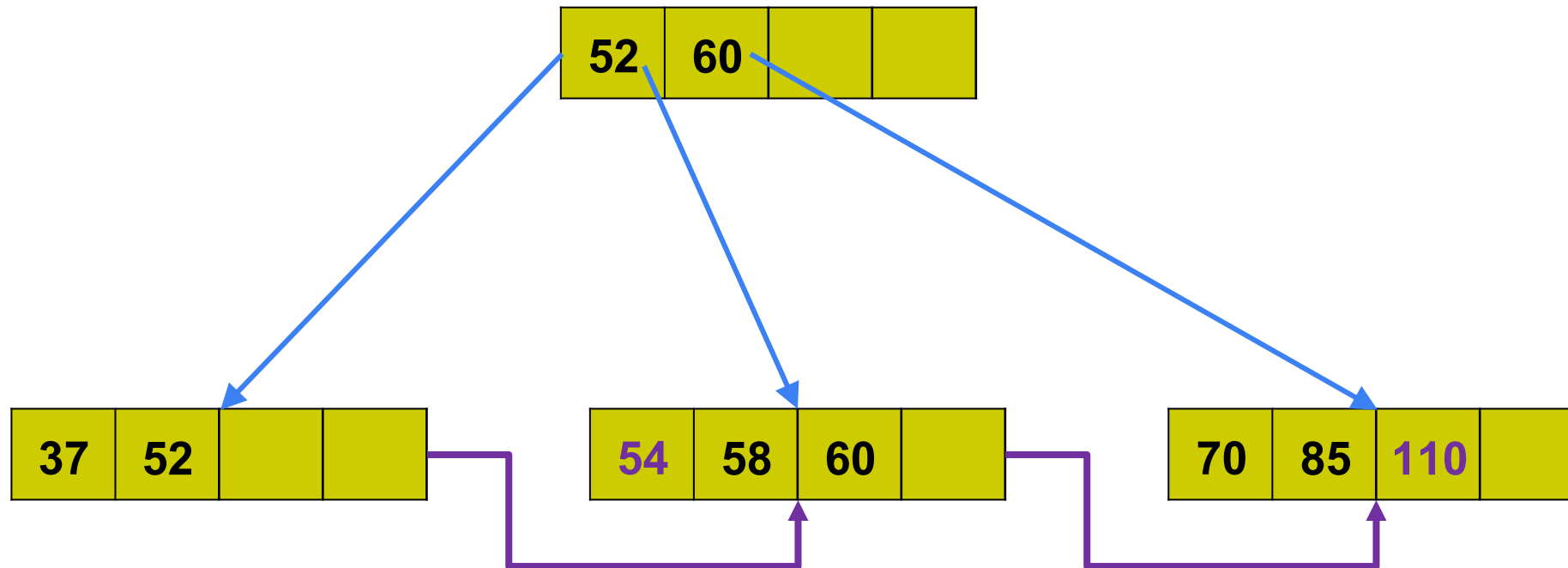
8. Árvore B+ - Inserção

- Exemplo: Inserir chave 37 (depois)



8. Árvore B+ - Inserção

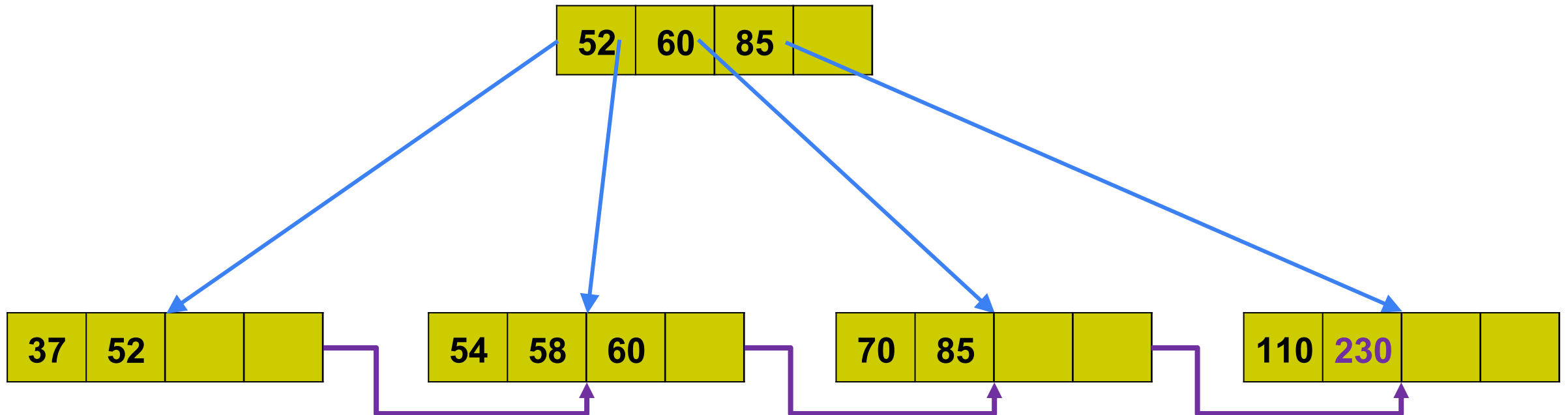
- Exemplo: Inserir chaves 54, 110 (depois)



- Inserir chave 230 → **Realizar cisão**

8. Árvore B+ - Inserção

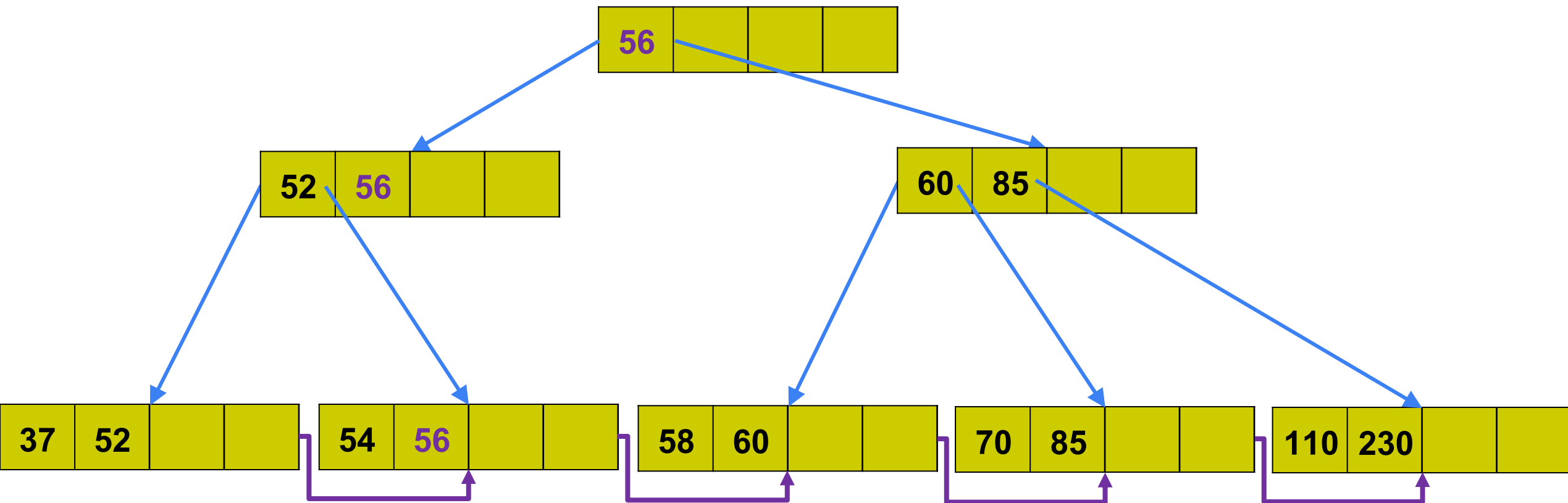
- Exemplo: Inserir chave 230 (depois)



- Inserir chave 56 → **Realizar cisão**

8. Árvore B+ - Inserção

- Exemplo: Inserir chave 56 (depois)



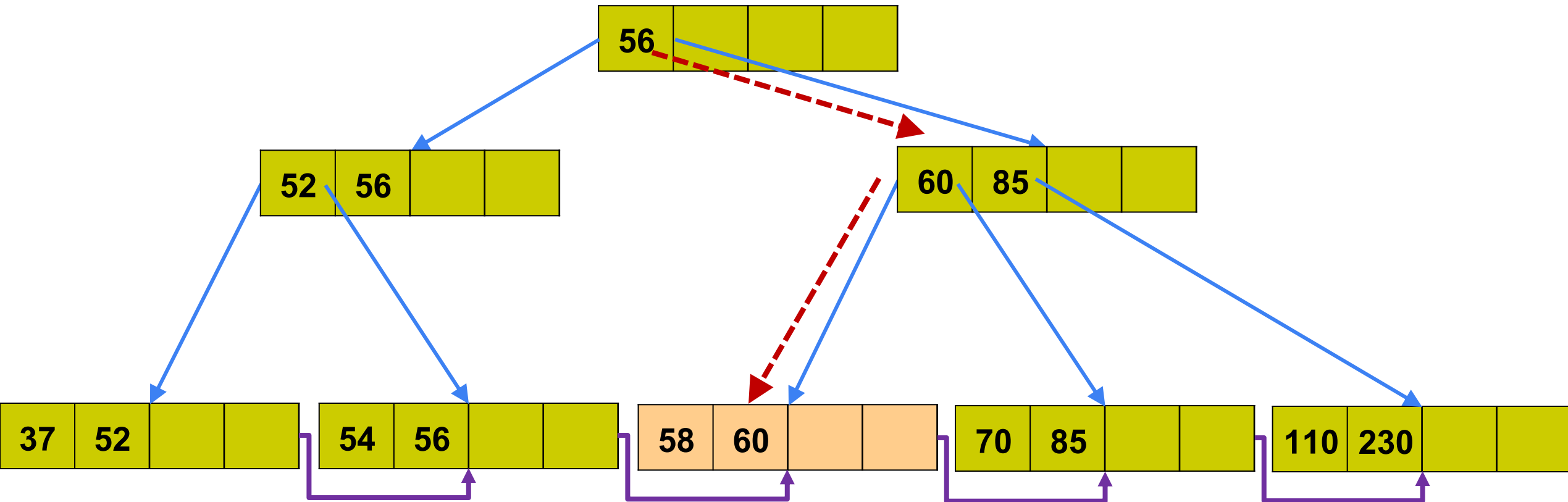


8. Árvore B+ - Pesquisa

- Semelhante à pesquisa em árvore B
- A pesquisa sempre leva a uma página folha
- A pesquisa **não para se a chave procurada for encontrada em uma página índice.**

8. Árvore B+ - Pesquisa

- Exemplo: Procurar a chave 60



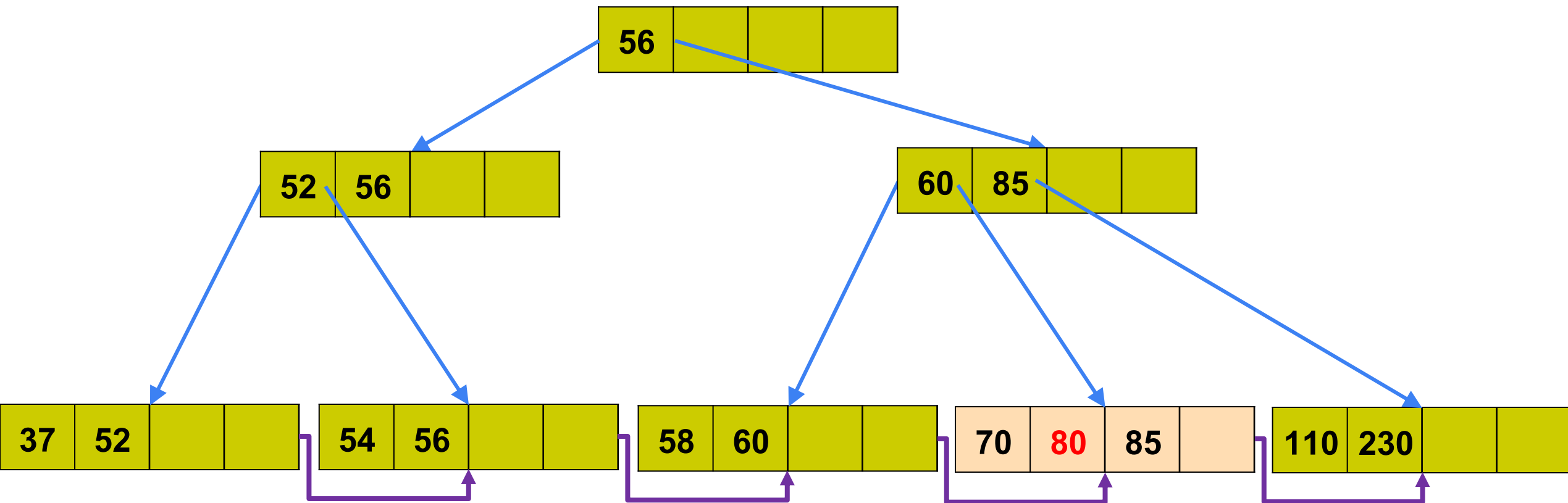


8. Árvore B+ - Remoção

- Remoção de uma chave **X**
 - **Caso 1:** A chave **X** aparece apenas em um nó folha
 - A chave **X** é simplesmente removida e a folha é reorganizada

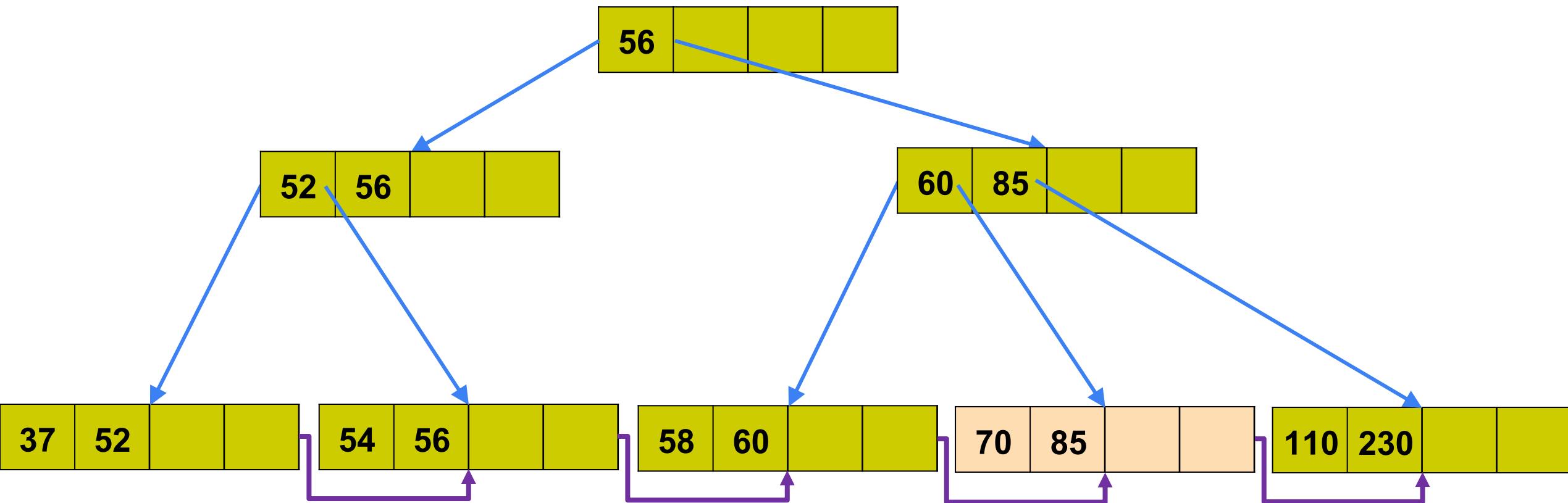
8. Árvore B+ - Remoção

- Exemplo: Remover chave 80 (antes)



8. Árvore B+ - Remoção

- Exemplo: Remover chave 80 (antes)



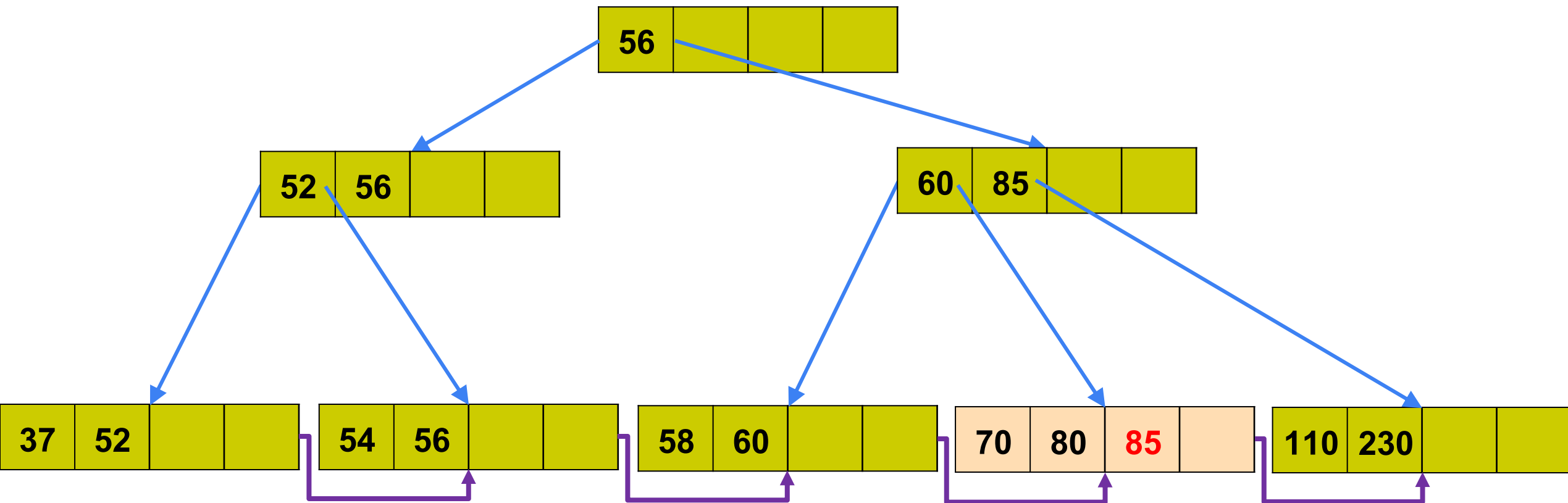


8. Árvore B+ - Remoção

- Remoção de uma chave **X**
 - **Caso 2:** A chave X aparece também em nós internos (índice)
 - A chave **X** é removida
 - A folha é reorganizada
 - A chave **X não** é removida dos nós internos

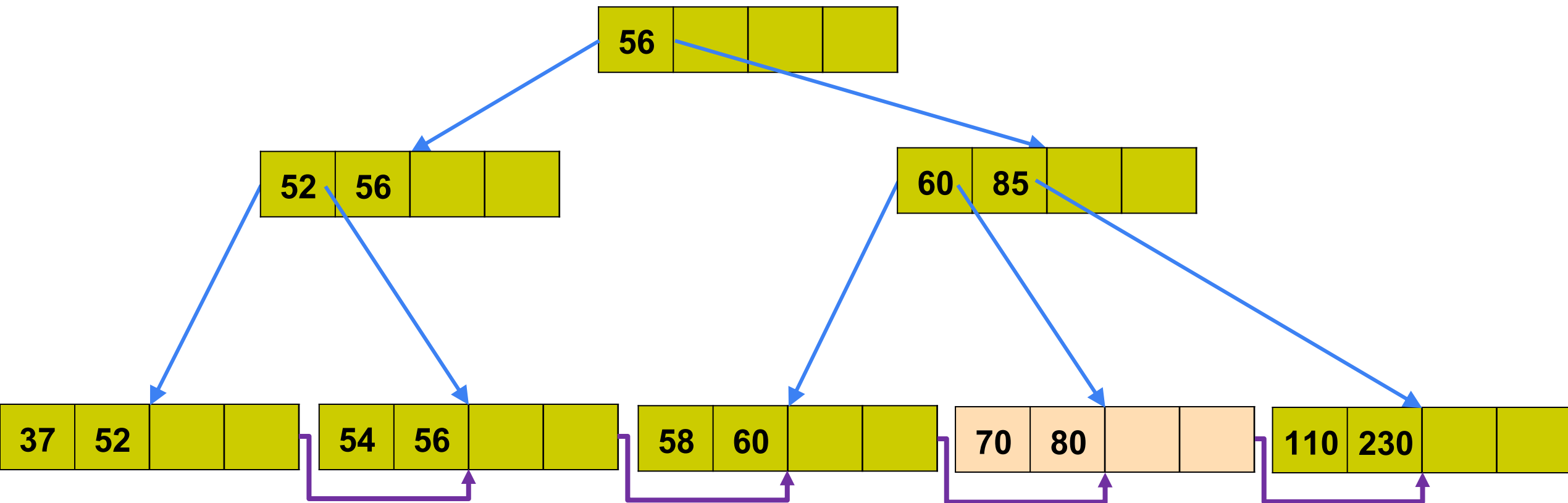
8. Árvore B+ - Remoção

- Exemplo: Remover chave 85 (antes)



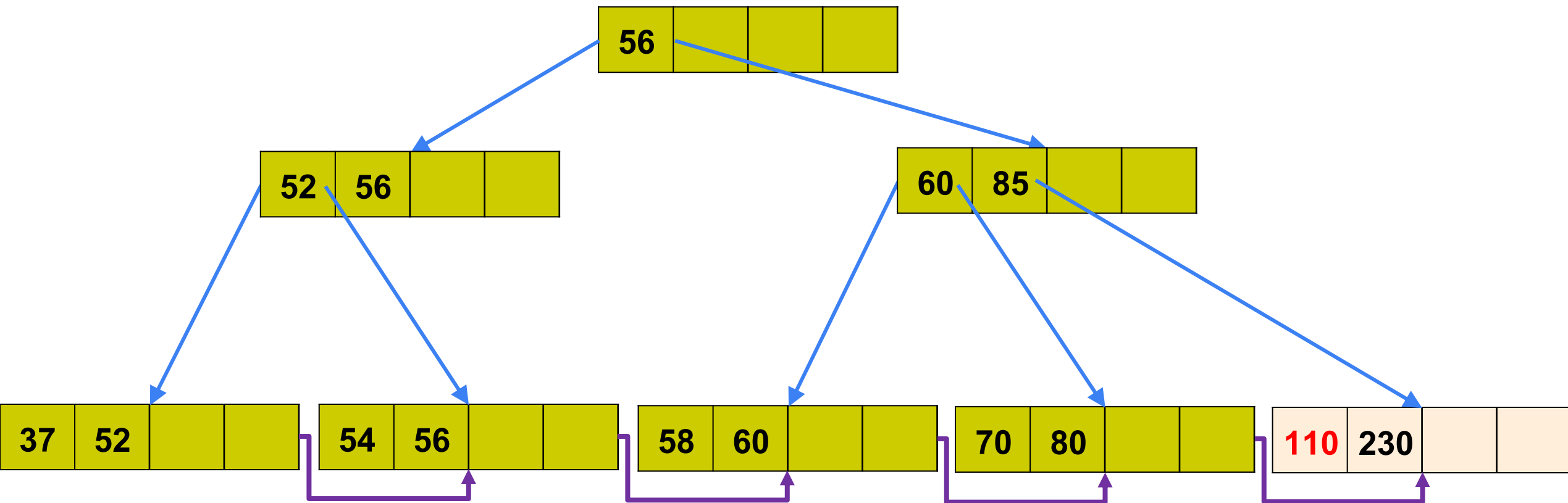
8. Árvore B+ - Remoção

- Exemplo: Remover chave 85 (depois)



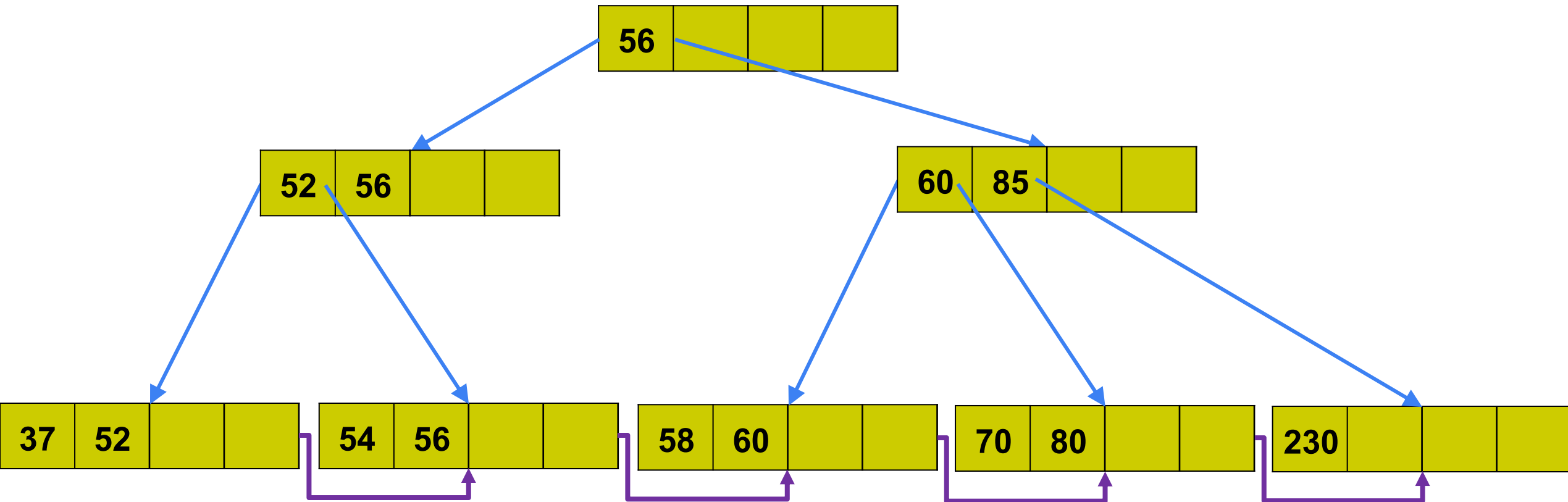
8. Árvore B+ - Remoção

- Exemplo: Remover chave 110 (antes)



8. Árvore B+ - Remoção

- Exemplo: Remover chave 110 (depois)





8. Árvore B+ - Remoção

- Quando uma chave é retirada de um nó folha, o número de chaves restantes pode ser menor que **$(M-1)/2$**
- Tratamentos:
 - Concatenação
 - Redistribuição



8. Árvore B+ - Remoção

- Remoção com Concatenação
 - Duas páginas **P** e **Q** são chamadas irmãos adjacentes se têm o mesmo pai **W** e são apontadas por ponteiros adjacentes em **W**
 - **P** e **Q** podem ser concatenadas se são irmãos adjacentes e juntas possuem menos de $M-1$ chaves.

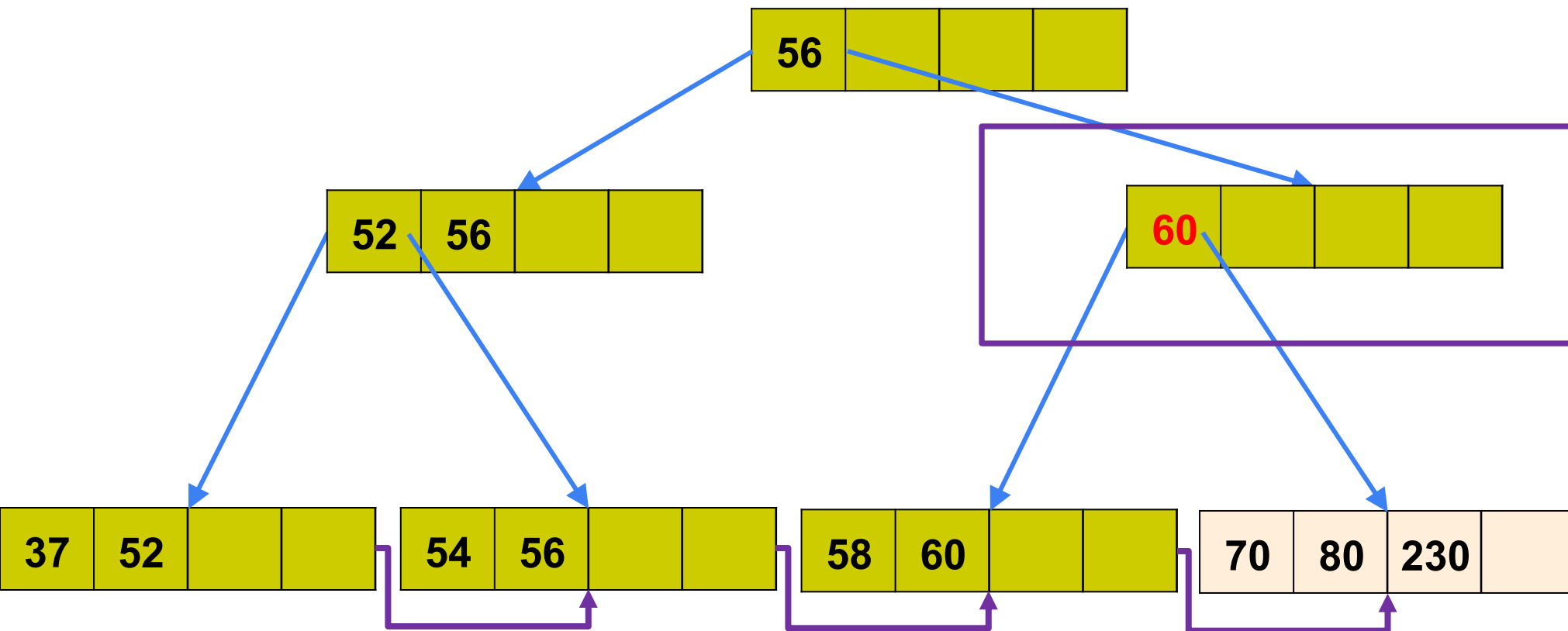


8. Árvore B+ - Remoção

- Remoção com Concatenação
 - A concatenação agrupa as entradas de duas páginas em uma só
 - No nó pai deixa de existir uma entrada: aquela da chave que se encontra entre os ponteiros para **P** e **Q**
 - Essa chave é simplesmente removida do nó pai

8. Árvore B+ - Remoção

- Exemplo: Remover (**concatenação**) chave 110 (depois)



8. Árvore B+ - Remoção

- Remoção com Concatenação
 - Como foi retirada uma chave do nó **W**, caso ele passe a ter menos de $(M-1)/2$ chaves, o processo se repete
 - Ou seja, a concatenação é um processo propagável
 - Se a propagação atingir a raiz, a árvore diminuirá de altura



8. Árvore B+ - Remoção

- Remoção com Redistribuição
 - Se a página **P** e seu irmão adjacente **Q** possuem em conjunto **M-1** ou mais chaves, estas podem ser equilibradamente distribuídas:
 - Concatena-se **P** e **Q**
 - Efetua-se a cisão da página resultante



8. Comparação B e B+

- Diferenças estruturais

Característica	Árvore B	Árvore B+
Onde ficam as chaves?	Em todos os nós (internos e folhas)	Somente nas folhas
Nós internos	Guardam chaves + ponteiros	Guardam somente índices
Folhas	Parecidas com nós internos	Armazenam todos os dados
Encadeamento entre folhas	✗ Não possui	✓ Sim, folhas ligadas como lista



8. Comparação B e B+

- Comportamento da Busca

Característica	B	B+
Local onde a busca pode terminar	Pode terminar em nó interno	Sempre termina na folha
Caminho até os dados	Direto (pode parar antes)	Sempre até a folha
Percorrer dados em ordem	$O(\log N)$ por passo	$O(1)$ por passo (via ponteiro para próxima folha)

8. Comparação B e B+

- Inserção e Remoção

Operação	Árvore B	Árvore B+
Inserção	Chave sobe ao pai e deixa a folha	Chave sobe ao pai, mas permanece na folha
Duplicação de chave	✗ Não ocorre	✓ Índices duplicados nos nós internos
Remoção	Remove de folha ou nó interno	Remove somente da folha
Atualização de índice	Pode alterar nó interno	Nem sempre é alterado



8. Comparação B e B+

- Aplicações e Vantagens
 - Árvore B
 - Estrutura completa: dados em qualquer nó
 - Boa para acesso aleatório equilibrado
 - Menos duplicação de informação
 - Árvore B+
 - Padrão de bancos de dados (SQL Server, Oracle)
 - Melhor para operações de intervalo (range query)
 - Menor altura (índices mais compactos)
 - Navegação sequencial extremamente rápida
 - Mais eficiente em acesso a disco



Observações sobre o material eletrônico

- O material ficará disponível na pasta compartilhada que é acessada sob convite
- O material foi elaborado a partir de diversas fontes (livros, internet, colegas, alunos etc.)
- Alguns trechos podem ter sido inteiramente transcritos a partir dessas fontes
- Outros trechos são de autoria própria
- Esta observação deve estar presente em qualquer utilização do material fora do ambiente de aulas do IFSP - Catanduva